ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
AUTONOMOUS INSTITUTION
Approved by AICTE & Affiliated to Anna University
NBA Accredited for BE (ECE, EEE, MECH) | Accredited by NAAC with A+ Grade
Anjugramam - Kanyakumari Main Road, Palkulam, Variyoor P.O. - 629 401, Kanyakumari District.

DEPARTMENT OF COMPUTER SCINECE AND ENGINEERING

24CS331 - PROGRAMMING FOR PROBLEM SOLVING USING PYTHON

LAB MANUAL

SEMESTER: I **REGULATION:2024**

SUB CODE: 24CS331

SUB NAME: Programming for problem solving using python

Prepared By

Mrs. R. Sahila Devi

Approved By:

Approved on:

Revision Number: **Revision Date:**

CONTENTS

SI.NO	DATE	NAME OF EXPERIMENTS	PAGE NO	MARKS	SIGNATURE
1.		DEVELOP SIMPLE PYTHON PROGRAMS USING BASIC DATA TYPES			
2.		DEVELOP SIMPLE PYTHON PROGRAMS USING OPERATORS AND EXPRESSIONS			
3.		DEVELOP PYTHON PROGRAMS USING CONDITIONAL STATEMENTS			
4(a).		FACTORIAL USING FOR LOOP			
4(b).		MULTIPLICATION TABLE USING WHILE AND DO-WHILE LOOP			
5.		DEVELOP PYTHON PROGRAMS USING FUNCTIONS			
6.		DEVELOP PROGRAMS TO DEMONSTRATE THE USE OF LISTS AND TUPLES			
7.		DEVELOP PROGRAMS TO DEMONSTRATE THE USE OF DICTIONARIES			
8.		DEMONSTRATE THE VARIOUS STRING MANIPULATION FUNCTIONS			
9.		DEVELOP PROGRAMS TO SHOW EXCEPTION HANDLING IN TASKS			
10.		EXECUTE PROGRAMS USING NUMPY IN JUPYTER NOTEBOOK			
11.		PYTHON PROGRAM USING FILE I/O, RANDOM ACCESS FILE HANDLING METHODS AND ZIPPING/UNZIPPING OF FILES			
12.		DEVELOP PYTHON PROGRAMS USING PACKAGES			

EXPERIMENT 1 — DEVELOP SIMPLE PYTHON PROGRAMS USING BASIC DATA TYPES

Aim:

Demonstrate Python basic data types (int, float, str, bool), and simple type conversions.

Algorithm / Steps:

1. Declare variables of different types.
2. Print values and types.
3. Perform conversions between int, float and str.
4. Show boolean expressions.

Program

```
# Experiment 1: Basic data types and conversion
a = 10              # int
b = 3.14            # float
c = "Hello"         # str
d = True            # bool

print("a:", a, "type:", type(a))
print("b:", b, "type:", type(b))
print("c:", c, "type:", type(c))
print("d:", d, "type:", type(d))

# conversions
a_to_float = float(a)
b_to_int = int(b)
num_str = str(a + int(b))

print("a to float:", a_to_float)
print("b to int:", b_to_int)
print("a+b as string:", num_str)

# boolean expression
print("Is a > b?", a > b)
```

Output

```
a: 10 type: <class 'int'>
b: 3.14 type: <class 'float'>
c: Hello type: <class 'str'>
d: True type: <class 'bool'>
a to float: 10.0
b to int: 3
a+b as string: 13
Is a > b? True
```

Viva Questions & Answers

1. Q: Name immutable basic Python types.
   A: `int`, `float`, `str`, `tuple`, `bool` are immutable.

2. Q: What does `int(3.9)` return?
   A: `3` — it truncates toward zero.
3. Q: How do you check the type of a variable?
   A: Use `type(variable)`.

Result:

Thus the program for develop simple python basic datatypes (int , float , str , bool) and simple type conversions is excecuted and the output verified successfully.

EXPERIMENT 2 — DEVELOP SIMPLE PYTHON PROGRAMS USING OPERATORS AND EXPRESSIONS

Aim:

Demonstrate arithmetic, relational, logical, assignment and bitwise operators.

Algorithm / Steps:

1. Read or define operands.
2. Apply arithmetic and relational operators.
3. Demonstrate logical operators.
4. Show bitwise operations and assignment operators.

Program

```
# Experiment 2: Operators and expressions
x = 7
y = 3

# arithmetic
print("x+y", x+y)
print("x-y", x-y)
print("x*y", x*y)
print("x/y", x/y)
print("x//y", x//y, "x%y", x%y, "x**y", x**y)

# relational
print("x > y:", x > y, "x == y:", x == y)

# logical
print("x>2 and y<5:", x>2 and y<5)
print("x==7 or y==10:", x==7 or y==10)
print("not (x==y):", not(x==y))

# bitwise
print("x & y:", x & y)
print("x | y:", x | y)
print("x ^ y:", x ^ y)
print("~x:", ~x)
```

Output

```
x+y 10
x-y 4
x*y 21
x/y 2.3333333333333335
x//y 2 x%y 1 x**y 343
x > y: True x == y: False
x>2 and y<5: True
x==7 or y==10: True
not (x==y): True
x & y: 3
x | y: 7
x ^ y: 4
~x: -8
```

Viva Questions & Answers

1. Q: Difference between `/` and `//`?
   A: `/` gives float division, `//` gives floor integer division.
2. Q: What is the result of `~x` for x=7?
   A: `~7` equals `-8` (because of two's complement).
3. Q: How does `and` short-circuit?
   A: If the left operand is False, the right is not evaluated.

Result:

Thus the program for develop simple python programs using operators and expressions is excecuted and the output verified successfully.

EXPERIMENT 3 — DEVELOP PYTHON PROGRAMS USING CONDITIONAL STATEMENTS

Aim:

Use `if`, `elif`, `else` and nested conditionals to implement logic (e.g., grade classification).

Algorithm / Steps:

1. Read numeric marks.
2. Use conditional checks to classify grade.
3. Print grade.

Program

```
# Experiment 3: Conditional statements - grade classifier
marks = int(input("Enter marks (0-100): "))

if marks < 0 or marks > 100:
    print("Invalid marks")
elif marks >= 90:
    print("Grade: A+")
elif marks >= 75:
    print("Grade: A")
elif marks >= 60:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Grade: F")
```

Sample Interaction / Output

```
Enter marks (0-100):
```

```
Enter marks (0-100): 40
Grade: F
```

Viva Questions & Answers

1. Q: Difference between `if` and `elif`?
   A: `elif` is checked only if previous `if`/`elif` were False; `if` is a fresh conditional.
2. Q: Can you write a `switch` statement in Python?
   A: No `switch` keyword; use `if-elif` or `match` (Python 3.10+).
3. Q: What is nested `if`?
   A: An `if` inside another `if` — used for multi-level decisions.

Result:

Thus the program for develop simple python program using conditional statements is excecuted and the output verified successfully.

Experiment 4(a): Factorial Using *for* Loop

Aim:

To write a Python program to find the factorial of a given number using a **for loop**.

Algorithm:

1. Start the program.
2. Read an integer value **n** from the user.
3. Initialize a variable **fact = 1**.
4. Use a **for loop** from 1 to n:
   - Multiply `fact = fact * i`
5. After the loop ends, `fact` will contain the factorial of the number.
6. Display the result.
7. End the program.

Program:

```
# Program to find factorial of a number using for loop

n = int(input("Enter a number: "))
fact = 1

for i in range(1, n + 1):
    fact = fact * i

print("Factorial of", n, "is", fact)
```

Sample Output:

```
Enter a number: 5
Factorial of 5 is 120
```

Viva Questions & Answers:

1. **Q:** What is a factorial?
   A: Factorial of a number n (written as n!) is the product of all positive integers from 1 to n.
2. **Q:** What is the value of 0! ?
   A: 0! = 1 (by definition).
3. **Q:** What loop is used in this program and why?
   A: A **for loop** is used because the number of iterations (1 to n) is known beforehand.
4. **Q:** What happens if the user enters a negative number?
   A: Factorial is not defined for negative numbers; the program should ideally check and handle this case.
5. **Q:** Can factorial be computed using recursion?
   A: Yes, factorial can also be calculated using a recursive function.

Result:

Thus the program for factorial using for loop is excecuted and the output verified successfully.

Experiment 4(b) : Multiplication Table Using while and do-while Loop

Aim:

To write Python programs to print the multiplication table of a given number using

1. **while loop**
2. **do-while equivalent loop**

Algorithm:

1. Start the program.
2. Read a number **n** from the user.
3. Initialize **i = 1**.
4. Using a loop, print `n × i` for values 1 to 10.
5. Increment **i** until 10 is reached.
6. End the program.

Program 1: Multiplication Table Using while Loop

```
# Multiplication table using while loop

n = int(input("Enter a number: "))

i = 1
while i <= 10:
    print(n, "x", i, "=", n * i)
    i += 1
```

Sample Output

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Viva Questions & Answers:

1. **Q:** Which loop is used to generate the multiplication table?
   A: A *for loop* (or while loop) can be used.
2. **Q:** What is the range used for the table?
   A: From 1 to 10.
3. **Q:** Can we generate tables up to 20?
   A: Yes, just change the range in the loop.

4. **Q:** What will `n * i` produce?
   A: It gives the product of the input number and the loop counter.
5. **Q:** Write the output of 3 × 7.
   A: 21.

Result:

Thus the program multiplication table using while and do-while loop is excecuted and the output verified successfully.

EXPERIMENT 5 — DEVELOP PYTHON PROGRAMS USING FUNCTIONS

Aim: Show function definition, parameters, return values, default args, `*args`, `**kwargs`, and recursion.

Algorithm / Steps:

1. Define a normal function and a recursive function.
2. Show default arguments and variable arguments.
3. Call functions and print outputs.

Program

```
# Experiment 5: Functions
def add(a, b=5):
    "Add two numbers with default second argument"
    return a + b

def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n-1)

def var_args(*args, **kwargs):
    return args, kwargs

print("add(3):", add(3))
print("add(3, 4):", add(3,4))
print("factorial(5):", factorial(5))
print("var_args:", var_args(1,2, x=10, y=20))
```

Sample Output

```
add(3): 8
add(3, 4): 7
factorial(5): 120
var_args: ((1, 2), {'x': 10, 'y': 20})
```

Viva Questions & Answers

1. Q: What is recursion?
   A: A function calling itself with a base case to stop recursion.
2. Q: Difference between positional and keyword arguments?
   A: Positional are by order; keyword specify parameter names.
3. Q: What are `*args` and `**kwargs`?
   A: `*args` collects extra positional arguments as a tuple; `**kwargs` collects keyword args as a dict.

Result:

Thus the program for develop python programs using functions is excecuted and the output verified successfully.

EXPERIMENT 6 — DEVELOP PROGRAMS TO DEMONSTRATE THE USE OF LISTS AND TUPLES

Aim:

Demonstrate creation, indexing, slicing, methods of lists and tuples, and differences (mutability).

Algorithm / Steps:

1. Create list and tuple.
2. Show indexing and slicing.
3. Perform list operations: append, insert, remove, pop, sort.
4. Show tuple packing/unpacking.

Program

```
# Experiment 6: Lists and Tuples
lst = [3, 1, 4, 1, 5]
tup = (10, 20, 30)

print("Original list:", lst)
lst.append(9)
lst.insert(1, 2)
print("After append & insert:", lst)
lst.remove(1)  # removes first occurrence
print("After remove 1:", lst)
print("Sorted list:", sorted(lst))
print("Pop:", lst.pop())

# tuple
print("Tuple:", tup)
a, b, c = tup
print("Unpacked:", a, b, c)

# slicing
print("lst[1:4]:", lst[1:4])
```

Output

```
Original list: [3, 1, 4, 1, 5]
After append & insert: [3, 2, 1, 4, 1, 5, 9]
After remove 1: [3, 2, 4, 1, 5, 9]
Sorted list: [1, 2, 3, 4, 5, 9]
Pop: 9
Tuple: (10, 20, 30)
Unpacked: 10 20 30
lst[1:4]: [2, 4, 1]
```

Viva Questions & Answers

1. Q: Are lists mutable? Tuples?
   A: Lists are mutable; tuples are immutable.

2. Q: How to convert list to tuple?
   A: Use `tuple(mylist)`.
3. Q: Which list method sorts in-place?
   A: `list.sort()` sorts in-place; `sorted(list)` returns a new list.

Result:

Thus the program for develop programs to demonstrate the use of lists and tuples is excecuted and the output verified successfully.

EXPERIMENT 7 — DEVELOP PROGRAMS TO DEMONSTRATE THE USE OF DICTIONARIES

Aim:

Create and manipulate dictionaries, iterate key/value pairs, and show dictionary comprehensions.

Algorithm / Steps:

1. Create dict, access, add, remove keys.
2. Iterate keys(), values(), items().
3. Use dictionary comprehension to build new dict.

Program

```
# Experiment 7: Dictionaries
student = {"id": 101, "name": "Anu", "marks": 82}
print("Student:", student)
print("Name:", student["name"])

# update/add
student["section"] = "C"
student["marks"] = 85
print("Updated:", student)

# iterate
for k, v in student.items():
    print(k, "->", v)

# frequency counting using dictionary
text = "apple banana apple mango"
freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1
print("Frequencies:", freq)

# comprehension
squares = {i: i*i for i in range(1,6)}
print("Squares:", squares)
```

Sample Output

```
Student: {'id': 101, 'name': 'Anu', 'marks': 82}
Name: Anu
Updated: {'id': 101, 'name': 'Anu', 'marks': 85, 'section': 'C'}
id -> 101
name -> Anu
marks -> 85
section -> C
Frequencies: {'apple': 2, 'banana': 1, 'mango': 1}
Squares: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Viva Questions & Answers

1. Q: How to safely access a key that might not exist?
   A: Use `dict.get(key, default)` or check `if key in dict`.
2. Q: Are dictionary keys ordered?
   A: From Python 3.7+, insertion order is preserved.
3. Q: Can a dict key be a list?
   A: No — keys must be hashable (immutable); lists are not hashable.

Result:

Thus the program for develop programs to demonstrate the use of dictionaries is excecuted and the output verified successfully.

EXPERIMENT 8 — DEMONSTRATE THE VARIOUS STRING MANIPULATION FUNCTIONS

Aim:

Use built-in string methods: `split`, `join`, `strip`, `lower`, `upper`, `replace`, `find`, formatting and f-strings.

Algorithm / Steps:

1. Show basic methods and index access.
2. Check palindrome.
3. Demonstrate formatting.

Program

```
# Experiment 8: String manipulations
s = "  Hello, Python Lab!  "
print("Original:", repr(s))
print("strip:", repr(s.strip()))
print("lower:", s.lower())
print("replace:", s.replace("Python", "World"))

words = "apple,banana, mango"
lst = [w.strip() for w in words.split(",")]
print("Split & strip:", lst)
joined = " | ".join(lst)
print("Joined:", joined)

# palindrome check
t = "Madam"
clean = ''.join(ch.lower() for ch in t if ch.isalnum())
is_pal = clean == clean[::-1]
print(f"'{t}' palindrome? {is_pal}")
```

Sample Output

```
Original: '  Hello, Python Lab!  '
strip: 'Hello, Python Lab!'
lower:   hello, python lab!
replace:   Hello, World Lab!
Split & strip: ['apple', 'banana', 'mango']
Joined: apple | banana | mango
'Madam' palindrome? True
```

Viva Questions & Answers

1. Q: How to remove leading and trailing spaces from a string?
   A: Use `str.strip()`.
2. Q: How to split a string into words?
   A: Use `str.split()` (default uses whitespace) or `split(sep)`.
3. Q: How do you format variables into strings?
   A: Use `f-strings` (e.g. `f"{var}"`), `str.format()`, or `%` formatting.

Result:

Thus the program for demonstrate the various string manipulation functions is excecuted and the output verified successfully.

EXPERIMENT 9 — DEVELOP PROGRAMS TO SHOW EXCEPTION HANDLING IN TASKS

Aim:

Demonstrate `try`, `except`, `else`, `finally`, and raise custom exceptions.

Algorithm / Steps:

1. Take input that may cause `ValueError` or `ZeroDivisionError`.
2. Use `try/except` to handle errors.
3. Use `finally` to run cleanup code.
4. Raise and catch a custom exception.

Program

```
# Experiment 9: Exception handling
def safe_divide():
    try:
        a = int(input("Enter numerator: "))
        b = int(input("Enter denominator: "))
        result = a / b
    except ValueError:
        print("Invalid input: please enter integers.")
    except ZeroDivisionError:
        print("Cannot divide by zero.")
    else:
        print("Result:", result)
    finally:
        print("Done safe_divide")

safe_divide()

# custom exception
class NegativeNumberError(Exception):
    pass

def check_positive(n):
    if n < 0:
        raise NegativeNumberError("Negative number not allowed")
    return n

try:
    print("check_positive(5):", check_positive(5))
    print("check_positive(-3):", check_positive(-3))
except NegativeNumberError as e:
    print("Caught:", e)
```

Sample Interaction / Output

```
Enter numerator: 10
Enter denominator: 0
Cannot divide by zero.
Done safe_divide
check_positive(5): 5
Caught: Negative number not allowed
```

Viva Questions & Answers

1. Q: When is `finally` executed?
   A: Always — whether exception occurs or not.
2. Q: How to define custom exceptions?
   A: Create a class that inherits from `Exception`.
3. Q: Difference between `except Exception as e` and bare `except:`?
   A: Bare `except:` catches all exceptions (including system-exiting), not recommended. `except Exception as e` is safer.

Result:

Thus the program for develop programs to show exception handling in tasks is excecuted and the output verified successfully.

EXPERIMENT 10 — EXECUTE PROGRAMS USING NUMPY IN JUPYTER NOTEBOOK

Aim:

Demonstrate NumPy array creation, indexing, vectorized operations and matrix multiplication.

Prerequisite: Install numpy (`pip install numpy`) and run in Jupyter for interactive display.

Algorithm / Steps:

1. Import numpy as `np`.
2. Create arrays, show shapes and dtypes.
3. Perform element-wise ops and matrix multiply.

Program

```
# Experiment 10: NumPy basics (run in Jupyter)
import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])
print("a:", a, "shape:", a.shape)
print("b:", b, "shape:", b.shape)

print("a + b:", a + b)
print("a * 2:", a * 2)
print("dot(a,b):", np.dot(a,b))

# 2D arrays
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print("A:\n", A)
print("B:\n", B)
print("A * B (elementwise):\n", A * B)
print("A dot B (matrix multiply):\n", A.dot(B))
```

Sample Output

```
a: [1 2 3] shape: (3,)
b: [4 5 6] shape: (3,)
a + b: [5 7 9]
a * 2: [2 4 6]
dot(a,b): 32
A:
 [[1 2]
 [3 4]]
B:
 [[5 6]
 [7 8]]
A * B (elementwise):
 [[ 5 12]
 [21 32]]
A dot B (matrix multiply):
 [[19 22]
```

```
[43 50]]
```

Viva Questions & Answers

1. Q: Why use NumPy arrays instead of Python lists?
   A: NumPy arrays provide efficient memory, vectorized operations and many numeric functions.
2. Q: How to get shape and dtype of an array?
   A: `arr.shape`, `arr.dtype`.
3. Q: Difference between `*` and `dot` for NumPy arrays?
   A: `*` is elementwise multiply; `dot` (or `@`) is matrix multiplication for 2D arrays.

Result:

Thus the program for execute programs using numpy in jupyter notebook is excecuted and the output verified successfully.

EXPERIMENT 11 — PYTHON PROGRAM USING FILE I/O, RANDOM ACCESS FILE HANDLING METHODS AND ZIPPING/UNZIPPING OF FILES

Aim:

Demonstrate reading/writing files, random access (`seek`, `tell`), using `random` module, and compressing/uncompressing with `zipfile`.

Algorithm / Steps:

1. Write text content to a file.
2. Read file contents, use `seek` and `tell`.
3. Use `random` to write random numbers.
4. Compress file with `zipfile` and then extract.

Program

```
# Experiment 11: File I/O, random access, zipping
import random
import zipfile
import os

fname = "sample.txt"
# write
with open(fname, "w") as f:
    f.write("Line1\nLine2\nLine3\n")

# append random numbers
with open(fname, "a") as f:
    for _ in range(3):
        f.write(str(random.randint(1,100)) + "\n")

# read and demonstrate seek/tell
with open(fname, "r") as f:
    print("Start pos:", f.tell())
    print("First line:", f.readline().strip())
    print("Pos after reading one line:", f.tell())
    f.seek(0)
    print("After seek to 0, read all:")
    print(f.read())

# zip the file
zip_name = "archive.zip"
with zipfile.ZipFile(zip_name, 'w') as z:
    z.write(fname)

print("Created zip:", zip_name)
# unzip to folder
with zipfile.ZipFile(zip_name, 'r') as z:
    z.extractall("extracted_files")
print("Extracted files:", os.listdir("extracted_files"))
```

Sample Output (random numbers will vary)

```
Start pos: 0
First line: Line1
```

```
Pos after reading one line: 6
After seek to 0, read all:
Line1
Line2
Line3
42
7
89

Created zip: archive.zip
Extracted files: ['sample.txt']
```

Viva Questions & Answers

1. Q: What does `f.seek(0)` do?
   A: Moves file pointer to the beginning of file (offset 0).
2. Q: How to write binary data to a file?
   A: Open file with mode `'wb'` and write bytes objects.
3. Q: How to compress files in Python?
   A: Use the `zipfile` module (`ZipFile`) or `shutil.make_archive`.

Result:

Thus the program for python program using file I/O , random access file handling methods and zipping/unzipping of files is excecuted and the output verified successfully.

EXPERIMENT 12 — DEVELOP PYTHON PROGRAMS USING PACKAGES

Aim:

Show module/package creation and usage, and how to import functions or modules. Also show using a third-party package.

Algorithm / Steps:

1. Create a Python module file `mymath.py` with functions.
2. Create a package folder structure if desired.
3. Import and use module functions.
4. Show how to install and use a third-party package (`requests` example — mention install).

Program (module + usage)
Create a file `mymath.py` in the same directory:

```
# mymath.py
def add(a, b):
    return a + b

def mul(a, b):
    return a * b
```

Then program to use it:

```
# Experiment 12: Using packages/modules
import mymath
from mymath import add

print("mymath.add(2,3):", mymath.add(2,3))
print("add(4,5):", add(4,5))

# Using a third-party package example (requests) - ensure installed
# import requests
# r = requests.get("https://httpbin.org/get")
# print("Status code:", r.status_code)
```

Sample Output

```
mymath.add(2,3): 5
add(4,5): 9
```

Viva Questions & Answers

1. Q: How do you create a package?
   A: Create a folder, add an `__init__.py` (can be empty), and place modules inside; import as `package.module`.
2. Q: How to install third-party packages?
   A: Use `pip install package-name`.

3. Q: Difference between `import module` and `from module import func`?
   A: `import module` requires `module.func()` call; `from module import func` imports func directly.

Result:

Thus the program for develop python programs using packages is excecuted and the output verified successfully.